

PART 1: TECHNICAL INDICATOR LOGIC

1.1 Multi-Timeframe Architecture

DAILY (1D)	→ Trend Bias / Key S/R / EMA 200 alignment
4HOUR (4H)	→ Market Structure / Order Blocks / BOS-CHoCH
1HOUR (1H)	→ Entry Trigger / FVG / RSI Divergence
15MIN (15m)	→ Precision Entry / Liquidity Sweep

Rule: A signal requires alignment from at least 3 timeframes.

1.2 Market Structure (BOS / CHoCH)

```
import numpy as np
import pandas as pd

class MarketStructure:
    """
    Identifies Break of Structure (BOS) and Change of Character (CHoCH)
    using swing highs/lows.
    """

    def __init__(self, left_bars: int = 5, right_bars: int = 5):
        self.left = left_bars
        self.right = right_bars

    def find_swing_highs(self, highs: pd.Series) -> pd.Series:
        """Returns boolean series where swing highs occur."""
        swing_highs = pd.Series(False, index=highs.index)
        for i in range(self.left, len(highs) - self.right):
            window_left = highs.iloc[i - self.left:i]
            window_right = highs.iloc[i + 1:i + self.right + 1]
            if highs.iloc[i] > window_left.max() and highs.iloc[i] > window_right.max():
                swing_highs.iloc[i] = True
        return swing_highs

    def find_swing_lows(self, lows: pd.Series) -> pd.Series:
        """Returns boolean series where swing lows occur."""
        swing_lows = pd.Series(False, index=lows.index)
        for i in range(self.left, len(lows) - self.right):
            window_left = lows.iloc[i - self.left:i]
```

```

        window_right = lows.iloc[i + 1:i + self.right + 1]
        if lows.iloc[i] < window_left.min() and lows.iloc[i] < window_right.min():
            swing_lows.iloc[i] = True
    return swing_lows

def detect_structure(self, df: pd.DataFrame) -> dict:
    """
    Returns current market structure state.
    """
    highs = df['high']
    lows = df['low']
    closes = df['close']

    swing_highs = self.find_swing_highs(highs)
    swing_lows = self.find_swing_lows(lows)

    # Get last significant swings
    sh_indices = swing_highs[swing_highs].index
    sl_indices = swing_lows[swing_lows].index

    if len(sh_indices) < 2 or len(sl_indices) < 2:
        return {"structure": "UNDEFINED", "trend": "NEUTRAL"}

    last_sh = highs[sh_indices[-1]]
    prev_sh = highs[sh_indices[-2]]
    last_sl = lows[sl_indices[-1]]
    prev_sl = lows[sl_indices[-2]]

    # Bullish BOS: Close above previous swing high
    # Bearish BOS: Close below previous swing low
    # CHoCH: Break of internal structure indicating trend change

    if closes.iloc[-1] > prev_sh:
        return {"structure": "BOS_BULLISH", "trend": "BULLISH",
                "last_swing_high": last_sh, "last_swing_low": last_sl}
    elif closes.iloc[-1] < prev_sl:
        return {"structure": "BOS_BEARISH", "trend": "BEARISH",
                "last_swing_high": last_sh, "last_swing_low": last_sl}
    elif last_sl > prev_sl and last_sh < prev_sh:
        return {"structure": "CHoCH_BULLISH", "trend": "BULLISH_WEAK",
                "last_swing_high": last_sh, "last_swing_low": last_sl}
    elif last_sl < prev_sl and last_sh > prev_sh:
        return {"structure": "CHoCH_BEARISH", "trend": "BEARISH_WEAK",
                "last_swing_high": last_sh, "last_swing_low": last_sl}
    else:
        return {"structure": "CONSOLIDATION", "trend": "NEUTRAL",
                "last_swing_high": last_sh, "last_swing_low": last_sl}

```

1.3 Order Blocks & Fair Value Gaps (FVG)

```
class OrderBlockDetector:
    """
    Detects Bullish and Bearish Order Blocks.

    Bullish OB: Last bearish candle before a strong bullish move (BOS).
    Bearish OB: Last bullish candle before a strong bearish move (BOS).
    """

    def __init__(self, min_impulse_pct: float = 0.5):
        self.min_impulse = min_impulse_pct / 100

    def detect_bullish_obs(self, df: pd.DataFrame) -> list:
        """Returns list of bullish order blocks."""
        obs = []
        for i in range(2, len(df) - 1):
            # Look for bearish candle followed by strong bullish impulse
            candle_i = df.iloc[i]
            candle_next = df.iloc[i + 1]

            if candle_i['close'] < candle_i['open']: # Bearish candle
                impulse = (candle_next['close'] - candle_next['open']) / candle_next['open']
                if impulse > self.min_impulse and candle_next['close'] > candle_i['high']:
                    ob = {
                        'type': 'BULLISH',
                        'index': i,
                        'high': candle_i['high'],
                        'low': candle_i['low'],
                        'open': candle_i['open'],
                        'close': candle_i['close'],
                        'mitigated': False
                    }
                    obs.append(ob)
        return obs

    def detect_bearish_obs(self, df: pd.DataFrame) -> list:
        """Returns list of bearish order blocks."""
        obs = []
        for i in range(2, len(df) - 1):
            candle_i = df.iloc[i]
            candle_next = df.iloc[i + 1]

            if candle_i['close'] > candle_i['open']: # Bullish candle
                impulse = (candle_next['open'] - candle_next['close']) / candle_next['open']
                if impulse > self.min_impulse and candle_next['close'] < candle_i['low']:
                    ob = {
                        'type': 'BEARISH',
                        'index': i,
                        'high': candle_i['high'],
                        'low': candle_i['low'],
```

```

        'open': candle_i['open'],
        'close': candle_i['close'],
        'mitigated': False
    }
    obs.append(ob)
return obs

def find_nearest_unmitigated(self, obs: list, current_price: float, direction: str) -> dict:
    """Find nearest unmitigated OB that price hasn't traded through."""
    if direction == 'LONG':
        candidates = [ob for ob in obs if ob['type'] == 'BULLISH' and not ob['mitigated']]
        candidates.sort(key=lambda x: x['high'], reverse=True)
        for ob in candidates:
            if current_price > ob['low'] and current_price < ob['high'] * 1.02:
                return ob
    else:
        candidates = [ob for ob in obs if ob['type'] == 'BEARISH' and not ob['mitigated']]
        candidates.sort(key=lambda x: x['low'])
        for ob in candidates:
            if current_price < ob['high'] and current_price > ob['low'] * 0.98:
                return ob
    return None

class FairValueGap:
    """
    Detects Fair Value Gaps (imbalances) in price action.
    A bullish FVG: Low[i] > High[i-2] (gap up)
    A bearish FVG: High[i] < Low[i-2] (gap down)
    """

    def detect_fvgs(self, df: pd.DataFrame) -> list:
        fvgs = []
        for i in range(2, len(df)):
            # Bullish FVG
            if df.iloc[i]['low'] > df.iloc[i-2]['high']:
                fvg = {
                    'type': 'BULLISH',
                    'top': df.iloc[i]['low'],
                    'bottom': df.iloc[i-2]['high'],
                    'index': i,
                    'filled': False
                }
                fvgs.append(fvg)

            # Bearish FVG
            elif df.iloc[i]['high'] < df.iloc[i-2]['low']:
                fvg = {
                    'type': 'BEARISH',
                    'top': df.iloc[i-2]['low'],
                    'bottom': df.iloc[i]['high'],

```

```

        'index': i,
        'filled': False
    }
    fvgs.append(fvg)
return fvgs

def is_price_in_fvg(self, fvgs: list, price: float, fvg_type: str) -> dict:
    """Check if current price is inside an unfilled FVG."""
    for fvg in fvgs:
        if fvg['type'] == fvg_type and not fvg['filled']:
            if fvg['bottom'] <= price <= fvg['top']:
                return fvg
    return None

```

1.4 RSI Divergence Detection

```

class RSIDivergence:
    """
    Detects regular and hidden RSI divergences.

    Regular Bullish Divergence: Price lower low, RSI higher low → reversal up
    Regular Bearish Divergence: Price higher high, RSI lower high → reversal down
    Hidden Bullish Divergence: Price higher low, RSI lower low → continuation up
    Hidden Bearish Divergence: Price lower high, RSI higher high → continuation down
    """

    def __init__(self, period: int = 14, lookback: int = 50):
        self.period = period
        self.lookback = lookback

    def calculate_rsi(self, closes: pd.Series) -> pd.Series:
        delta = closes.diff()
        gain = delta.where(delta > 0, 0)
        loss = -delta.where(delta < 0, 0)

        avg_gain = gain.ewm(alpha=1/self.period, min_periods=self.period).mean()
        avg_loss = loss.ewm(alpha=1/self.period, min_periods=self.period).mean()

        rs = avg_gain / avg_loss
        rsi = 100 - (100 / (1 + rs))
        return rsi

    def find_peaks_and_troughs(self, series: pd.Series, order: int = 3) -> tuple:
        """Find local peaks and troughs in a series."""
        from scipy.signal import argrelextrema

        peaks_idx = argrelextrema(series.values, np.greater, order=order)[0]
        troughs_idx = argrelextrema(series.values, np.less, order=order)[0]

```

```

peaks = series.iloc[peaks_idx]
troughs = series.iloc[troughs_idx]

return peaks, troughs

def detect_regular_bullish(self, closes: pd.Series, rsi: pd.Series) -> bool:
    """Price makes lower low, RSI makes higher low."""
    price_troughs, _ = self.find_peaks_and_troughs(closes)
    rsi_troughs, _ = self.find_peaks_and_troughs(rsi)

    if len(price_troughs) < 2 or len(rsi_troughs) < 2:
        return False

    # Compare last two troughs
    p_ll = price_troughs.iloc[-1] < price_troughs.iloc[-2]
    r_hl = rsi_troughs.iloc[-1] > rsi_troughs.iloc[-2]

    return p_ll and r_hl and rsi_troughs.iloc[-1] < 40

def detect_regular_bearish(self, closes: pd.Series, rsi: pd.Series) -> bool:
    """Price makes higher high, RSI makes lower high."""
    _, price_peaks = self.find_peaks_and_troughs(closes)
    _, rsi_peaks = self.find_peaks_and_troughs(rsi)

    if len(price_peaks) < 2 or len(rsi_peaks) < 2:
        return False

    p_hh = price_peaks.iloc[-1] > price_peaks.iloc[-2]
    r_lh = rsi_peaks.iloc[-1] < rsi_peaks.iloc[-2]

    return p_hh and r_lh and rsi_peaks.iloc[-1] > 60

def detect_hidden_bullish(self, closes: pd.Series, rsi: pd.Series) -> bool:
    """Price makes higher low, RSI makes lower low."""
    price_troughs, _ = self.find_peaks_and_troughs(closes)
    rsi_troughs, _ = self.find_peaks_and_troughs(rsi)

    if len(price_troughs) < 2 or len(rsi_troughs) < 2:
        return False

    p_hl = price_troughs.iloc[-1] > price_troughs.iloc[-2]
    r_ll = rsi_troughs.iloc[-1] < rsi_troughs.iloc[-2]

    return p_hl and r_ll

def detect_hidden_bearish(self, closes: pd.Series, rsi: pd.Series) -> bool:
    """Price makes lower high, RSI makes higher high."""
    _, price_peaks = self.find_peaks_and_troughs(closes)
    _, rsi_peaks = self.find_peaks_and_troughs(rsi)

    if len(price_peaks) < 2 or len(rsi_peaks) < 2:

```

```

        return False

    p_lh = price_peaks.iloc[-1] < price_peaks.iloc[-2]
    r_hh = rsi_peaks.iloc[-1] > rsi_peaks.iloc[-2]

    return p_lh and r_hh

```

1.5 Volume Profile & Key Levels

```

class VolumeProfile:
    """
    Calculates Volume Profile to find Point of Control (POC)
    and Value Area High/Low.
    """

    def __init__(self, num_bins: int = 50):
        self.num_bins = num_bins

    def calculate(self, df: pd.DataFrame) -> dict:
        """Returns POC, VAH, VAL from recent price action."""
        price_range = np.linspace(df['low'].min(), df['high'].max(), self.num_bins)
        volume_profile = np.zeros(self.num_bins - 1)

        for i in range(len(df)):
            low_idx = np.searchsorted(price_range, df.iloc[i]['low']) - 1
            high_idx = np.searchsorted(price_range, df.iloc[i]['high'])
            low_idx = max(0, low_idx)
            high_idx = min(len(volume_profile) - 1, high_idx)

            for j in range(low_idx, high_idx + 1):
                volume_profile[j] += df.iloc[i]['volume']

        poc_idx = np.argmax(volume_profile)
        poc = (price_range[poc_idx] + price_range[poc_idx + 1]) / 2

        total_volume = volume_profile.sum()
        cumulative = 0
        vah_idx = val_idx = poc_idx

        # Value Area = 70% of volume
        for i in range(len(volume_profile)):
            idx = poc_idx + i if i % 2 == 0 else poc_idx - i
            if 0 <= idx < len(volume_profile):
                cumulative += volume_profile[idx]
                if cumulative / total_volume >= 0.7:
                    vah_idx = max(poc_idx, idx)
                    val_idx = min(poc_idx, idx)
                    break

```

```

    vah = (price_range[vah_idx] + price_range[vah_idx + 1]) / 2
    val = (price_range[val_idx] + price_range[val_idx + 1]) / 2

    return {
        'poc': poc,
        'vah': vah,
        'val': val,
        'profile': volume_profile
    }

def is_price_at_poc(self, price: float, profile: dict, tolerance: float = 0.005) -> bool:
    """Check if price is near Point of Control (high volume node)."""
    return abs(price - profile['poc']) / price < tolerance

```

1.6 Complete Confluence Engine

```

class ConfluenceEngine:
    """
    Combines all technical factors into a confidence score.
    Only generates signals when sufficient confluence exists.
    """

    def __init__(self):
        self.structure = MarketStructure()
        self.ob_detector = OrderBlockDetector()
        self.fvg_detector = FairValueGap()
        self.rsi_div = RSIDivergence()
        self.vol_profile = VolumeProfile()

    def analyze(self, df_4h: pd.DataFrame, df_1h: pd.DataFrame,
                current_price: float, funding_rate: float,
                open_interest_change: float) -> dict:
        """
        Full multi-timeframe analysis.
        Returns signal dict or None if no valid setup.
        """
        score = 0
        factors = []

        # === 4H STRUCTURE ===
        struct_4h = self.structure.detect_structure(df_4h)

        if struct_4h['trend'] == 'BULLISH':
            score += 20
            factors.append("4H Bullish BOS")
        elif struct_4h['trend'] == 'BEARISH':
            score += 20
            factors.append("4H Bearish BOS")

```

```

# === ORDER BLOCKS ===
bull_obs = self.ob_detector.detect_bullish_obs(df_4h)
bear_obs = self.ob_detector.detect_bearish_obs(df_4h)

nearest_bull = self.ob_detector.find_nearest_unmitigated(bull_obs, current_price, 'LONG')
nearest_bear = self.ob_detector.find_nearest_unmitigated(bear_obs, current_price, 'SHORT')

if nearest_bull and struct_4h['trend'] in ['BULLISH', 'BULLISH_WEAK']:
    score += 20
    factors.append(f"Bullish OB at {nearest_bull['low']:.2f}")

if nearest_bear and struct_4h['trend'] in ['BEARISH', 'BEARISH_WEAK']:
    score += 20
    factors.append(f"Bearish OB at {nearest_bear['high']:.2f}")

# === FAIR VALUE GAPS ===
fvg_1h = self.fvg_detector.detect_fvgs(df_1h)

if struct_4h['trend'] in ['BULLISH', 'BULLISH_WEAK']:
    fvg = self.fvg_detector.is_price_in_fvg(fvg_1h, current_price, 'BULLISH')
    if fvg:
        score += 15
        factors.append("Price in Bullish FVG")

if struct_4h['trend'] in ['BEARISH', 'BEARISH_WEAK']:
    fvg = self.fvg_detector.is_price_in_fvg(fvg_1h, current_price, 'BEARISH')
    if fvg:
        score += 15
        factors.append("Price in Bearish FVG")

# === RSI DIVERGENCE ===
rsi = self.rsi_div.calculate_rsi(df_1h['close'])

if self.rsi_div.detect_regular_bullish(df_1h['close'], rsi):
    score += 15
    factors.append("RSI Regular Bullish Divergence")
elif self.rsi_div.detect_hidden_bullish(df_1h['close'], rsi):
    score += 10
    factors.append("RSI Hidden Bullish Divergence")

if self.rsi_div.detect_regular_bearish(df_1h['close'], rsi):
    score += 15
    factors.append("RSI Regular Bearish Divergence")
elif self.rsi_div.detect_hidden_bearish(df_1h['close'], rsi):
    score += 10
    factors.append("RSI Hidden Bearish Divergence")

# === VOLUME PROFILE ===
vp = self.vol_profile.calculate(df_4h)
if self.vol_profile.is_price_at_poc(current_price, vp):
    score += 10

```

```

        factors.append("Price at Volume POC")

# === FUNDING RATE ===
if funding_rate < -0.01: # Very negative = long opportunity
    score += 10
    factors.append(f"Negative Funding ({funding_rate:.4f})")
elif funding_rate > 0.01: # Very positive = short opportunity
    score += 10
    factors.append(f"High Funding ({funding_rate:.4f})")

# === OPEN INTEREST ===
if open_interest_change > 5: # 5% increase
    score += 10
    factors.append(f"OI Rising ({open_interest_change:.1f}%)")

# === DETERMINE SIGNAL ===
if score >= 70:
    direction = 'LONG' if struct_4h['trend'] in ['BULLISH', 'BULLISH_WEAK'] else 'SHORT'

# Calculate levels
if direction == 'LONG':
    entry = current_price
    sl = nearest_bull['low'] * 0.995 if nearest_bull else current_price * 0.97
    tp1 = current_price + (current_price - sl) * 2
    tp2 = current_price + (current_price - sl) * 4
    tp3 = current_price + (current_price - sl) * 6
else:
    entry = current_price
    sl = nearest_bear['high'] * 1.005 if nearest_bear else current_price * 1.03
    tp1 = current_price - (sl - current_price) * 2
    tp2 = current_price - (sl - current_price) * 4
    tp3 = current_price - (sl - current_price) * 6

return {
    'valid': True,
    'direction': direction,
    'confidence': min(score, 100),
    'entry': entry,
    'stop_loss': sl,
    'take_profits': [
        (tp1, 50), # Close 50% at 1:2
        (tp2, 25), # Close 25% at 1:4
        (tp3, 25) # Trail 25% at 1:6
    ],
    'factors': factors,
    'timeframe': '4H/1H',
    'structure': struct_4h['structure']
}

return {'valid': False, 'score': score, 'factors': factors}

```

PART 2: EXCHANGE API INTEGRATION

2.1 Architecture Overview

EXCHANGE MANAGER
REST API → Historical data, account info, place orders
WebSocket → Real-time price, orderbook, liquidations
Rate Limiter → Prevent IP bans (1200 req/min Binance)
Retry Logic → Exponential backoff on failures

2.2 Exchange Manager (Production-Ready)

```
import ccxt
import asyncio
import websockets
import json
import time
from datetime import datetime, timedelta
from typing import Optional, Dict, List, Callable
import logging

logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)

class ExchangeManager:
    """
    Unified exchange manager supporting Binance, Bybit, and OKX.
    Handles both REST and WebSocket connections with robust error handling.
    """

    def __init__(self, exchange_id: str, api_key: str, api_secret: str,
                 sandbox: bool = True, passphrase: Optional[str] = None):
        self.exchange_id = exchange_id
        self.sandbox = sandbox

        # Initialize ccxt exchange
        exchange_class = getattr(ccxt, exchange_id)
        config = {
            'apiKey': api_key,
            'secret': api_secret,
            'enableRateLimit': True, # Built-in rate limiting
            'options': {
                'defaultType': 'swap', # Futures/perpetuals
                'adjustForTimeDifference': True,
            }
        }
```

```

    }

    if passphrase and exchange_id in ['okx', 'okex5']:
        config['password'] = passphrase

    self.exchange = exchange_class(config)

    if sandbox:
        self.exchange.set_sandbox_mode(True)

    # Rate limiting
    self.last_request_time = 0
    self.min_interval = 0.05 # 50ms between requests

    # WebSocket state
    self.ws_connections = {}
    self.price_callbacks = []
    self.liquidation_callbacks = []
    self.running = False

    # Cache
    self.price_cache = {}
    self.funding_cache = {}
    self.oi_cache = {}

    async def _rate_limited_request(self, method: str, *args, **kwargs):
        """Execute request with rate limiting."""
        elapsed = time.time() - self.last_request_time
        if elapsed < self.min_interval:
            await asyncio.sleep(self.min_interval - elapsed)

        try:
            self.last_request_time = time.time()
            result = await getattr(self.exchange, method)(*args, **kwargs)
            return result
        except ccxt.RateLimitExceeded:
            logger.warning("Rate limit hit, backing off...")
            await asyncio.sleep(2)
            return await self._rate_limited_request(method, *args, **kwargs)
        except ccxt.NetworkError as e:
            logger.error(f"Network error: {e}")
            await asyncio.sleep(1)
            return await self._rate_limited_request(method, *args, **kwargs)
        except ccxt.ExchangeError as e:
            logger.error(f"Exchange error: {e}")
            raise

    # ===== MARKET DATA =====

    async def fetch_ohlcv(self, symbol: str, timeframe: str = '1h',
                        limit: int = 500) -> List[list]:

```

```

        """Fetch candlestick data."""
        return await self._rate_limited_request(
            'fetch_ohlcv', symbol, timeframe, None, limit
        )

    async def fetch_ticker(self, symbol: str) -> dict:
        """Fetch current price and 24h stats."""
        return await self._rate_limited_request('fetch_ticker', symbol)

    async def fetch_order_book(self, symbol: str, limit: int = 50) -> dict:
        """Fetch order book (bid/ask)."""
        return await self._rate_limited_request('fetch_order_book', symbol, limit)

# ===== FUNDING & OI =====

    async def fetch_funding_rate(self, symbol: str) -> dict:
        """Fetch current funding rate."""
        if symbol in self.funding_cache:
            cache_time, rate = self.funding_cache[symbol]
            if time.time() - cache_time < 300: # 5 min cache
                return rate

        try:
            if self.exchange_id == 'binance':
                result = await self._rate_limited_request(
                    'fetch_funding_rate', symbol
                )
            elif self.exchange_id == 'bybit':
                result = await self._rate_limited_request(
                    'public_get_v5_market_tickers', {'category': 'linear', 'symbol': symbol}
                )
                # Parse Bybit response
                result = {'fundingRate': float(result['result']['list'][0]['fundingRate'])}
            else:
                result = await self._rate_limited_request(
                    'fetch_funding_rate', symbol
                )

            self.funding_cache[symbol] = (time.time(), result)
            return result
        except Exception as e:
            logger.error(f"Funding rate fetch failed: {e}")
            return {'fundingRate': 0}

    async def fetch_open_interest(self, symbol: str) -> dict:
        """Fetch open interest data."""
        try:
            if self.exchange_id == 'binance':
                # Binance OI endpoint
                self.exchange.load_markets()
                market = self.exchange.market(symbol)

```

```

        response = await self._rate_limited_request(
            'fapiPublic_get_open_interest', {'symbol': market['id']}
        )
        return {
            'openInterest': float(response['openInterest']),
            'timestamp': response['time']
        }
    elif self.exchange_id == 'bybit':
        response = await self._rate_limited_request(
            'public_get_v5_market_tickers', {'category': 'linear', 'symbol': symbol}
        )
        return {
            'openInterest': float(response['result']['list'][0]['openInterest']),
            'timestamp': time.time() * 1000
        }
    except Exception as e:
        logger.error(f"OI fetch failed: {e}")
        return {'openInterest': 0, 'timestamp': 0}

async def fetch_liquidations(self, symbol: str, since: Optional[int] = None,
                             limit: int = 100) -> List[dict]:
    """Fetch recent liquidation data."""
    try:
        if self.exchange_id == 'binance':
            params = {'symbol': symbol.replace('/', '').replace(':', ''),
                      'limit': limit}
            if since:
                params['startTime'] = since

            response = await self._rate_limited_request(
                'fapiPublic_get_force_orders', params
            )
            return response
        else:
            return []
    except Exception as e:
        logger.error(f"Liquidation fetch failed: {e}")
        return []

# ===== WEBSOCKET =====

async def start_websocket(self, symbols: List[str]):
    """Start WebSocket connections for real-time data."""
    self.running = True

    if self.exchange_id == 'binance':
        ws_url = "wss://fstream.binance.com/ws"
        streams = "/".join([f"{s.lower().replace('/', '').replace(':', '')}@kline_1m"
                             for s in symbols])
        full_url = f"{ws_url}/{streams}"

```

```

        asyncio.create_task(self._ws_price_handler(full_url, symbols))

    # Liquidation stream
    liq_url = f"{ws_url}/!forceOrder@arr"
    asyncio.create_task(self._ws_liquidation_handler(liq_url))

async def _ws_price_handler(self, url: str, symbols: List[str]):
    """Handle WebSocket price updates."""
    while self.running:
        try:
            async with websockets.connect(url) as ws:
                logger.info(f"Price WebSocket connected: {url}")
                async for message in ws:
                    data = json.loads(message)
                    if 'k' in data:
                        symbol = data['s']
                        price = float(data['k']['c'])
                        self.price_cache[symbol] = {
                            'price': price,
                            'timestamp': data['E']
                        }
                        for callback in self.price_callbacks:
                            await callback(symbol, price)
        except Exception as e:
            logger.error(f"Price WS error: {e}, reconnecting...")
            await asyncio.sleep(5)

async def _ws_liquidation_handler(self, url: str):
    """Handle liquidation stream."""
    while self.running:
        try:
            async with websockets.connect(url) as ws:
                logger.info("Liquidation WebSocket connected")
                async for message in ws:
                    data = json.loads(message)
                    if 'o' in data:
                        liq = data['o']
                        for callback in self.liquidation_callbacks:
                            await callback({
                                'symbol': liq['s'],
                                'side': liq['S'],
                                'quantity': float(liq['q']),
                                'price': float(liq['p']),
                                'timestamp': data['E']
                            })
        except Exception as e:
            logger.error(f"Liquidation WS error: {e}, reconnecting...")
            await asyncio.sleep(5)

def on_price_update(self, callback: Callable):
    """Register price update callback."""

```

```

        self.price_callbacks.append(callback)

    def on_liquidation(self, callback: Callable):
        """Register liquidation callback."""
        self.liquidation_callbacks.append(callback)

    async def stop(self):
        """Gracefully stop all connections."""
        self.running = False
        await self.exchange.close()
        logger.info("Exchange manager stopped")

# ===== USAGE EXAMPLE =====

async def main():
    # Initialize (use testnet keys!)
    manager = ExchangeManager(
        exchange_id='binance',
        api_key='YOUR_TESTNET_API_KEY',
        api_secret='YOUR_TESTNET_SECRET',
        sandbox=True
    )

    # Fetch historical data
    ohlcv = await manager.fetch_ohlcv('BTC/USDT:USDT', '4h', 500)
    df = pd.DataFrame(ohlcv, columns=['timestamp', 'open', 'high', 'low', 'close', 'volume'])

    # Fetch funding
    funding = await manager.fetch_funding_rate('BTC/USDT:USDT')
    print(f"Funding Rate: {funding['fundingRate']}")

    # Fetch OI
    oi = await manager.fetch_open_interest('BTC/USDT:USDT')
    print(f"Open Interest: {oi['openInterest']}")

    # Start WebSocket
    async def on_price(symbol, price):
        print(f"{symbol}: ${price}")

    manager.on_price_update(on_price)
    await manager.start_websocket(['BTC/USDT:USDT', 'ETH/USDT:USDT'])

    # Keep running
    await asyncio.sleep(3600)
    await manager.stop()

# asyncio.run(main())

```

2.3 Gold (XAU/USD) Data Integration

```
class GoldDataProvider:
    """
    Gold requires different data sources since most crypto exchanges
    don't offer XAU/USD. Use forex brokers or CFD platforms.
    """

    def __init__(self, oanda_token: Optional[str] = None):
        self.oanda_token = oanda_token
        self.base_url = "https://api-fxpractice.oanda.com" if oanda_token else None

    async def fetch_xau_usd(self, granularity: str = "H1", count: int = 500) -> pd.DataFrame:
        """Fetch XAU/USD from OANDA practice API."""
        if not self.oanda_token:
            # Fallback: Use Yahoo Finance via yfinance
            import yfinance as yf
            data = yf.download("GC=F", period="60d", interval="1h")
            data.columns = ['open', 'high', 'low', 'close', 'adj_close', 'volume']
            return data.reset_index()

        import aiohttp
        headers = {"Authorization": f"Bearer {self.oanda_token}"}
        url = f"{self.base_url}/v3/instruments/XAU_USD/candles"
        params = {
            "granularity": granularity,
            "count": count,
            "price": "M" # Midpoint
        }

        async with aiohttp.ClientSession() as session:
            async with session.get(url, headers=headers, params=params) as resp:
                data = await resp.json()

                candles = []
                for c in data.get('candles', []):
                    candles.append({
                        'timestamp': c['time'],
                        'open': float(c['mid']['o']),
                        'high': float(c['mid']['h']),
                        'low': float(c['mid']['l']),
                        'close': float(c['mid']['c']),
                        'volume': c['volume']
                    })

                return pd.DataFrame(candles)

    async def fetch_dxy_correlation(self) -> float:
        """Fetch DXY for inverse correlation check with Gold."""
        import yfinance as yf
        dxy = yf.download("DX-Y.NYB", period="5d", interval="1h")['Close']
```

```
xau = yf.download("GC=F", period="5d", interval="1h")['Close']

# Calculate rolling correlation
correlation = xau.corr(dxy)
return correlation
```

PART 3: TELEGRAM BOT SETUP

3.1 Architecture

TELEGRAM BOT
Polling / Webhook → Receive commands & send alerts
Signal Formatter → Beautiful, actionable messages
Inline Keyboard → Quick actions (Confirm, Skip, Details)
Alert Throttle → Prevent spam, batch similar alerts
User Management → Whitelist, subscription tiers

3.2 Complete Telegram Bot

```
import asyncio
import logging
from datetime import datetime
from typing import Dict, List, Optional
from dataclasses import dataclass

from telegram import Update, InlineKeyboardButton, InlineKeyboardMarkup
from telegram.ext import (
    Application, CommandHandler, CallbackQueryHandler,
    ContextTypes, MessageHandler, filters
)

logging.basicConfig(
    format='%(asctime)s - %(name)s - %(levelname)s - %(message)s',
    level=logging.INFO
)
logger = logging.getLogger(__name__)

@dataclass
class Signal:
    asset: str
    direction: str # LONG or SHORT
    entry: float
    entry_zone: tuple
    stop_loss: float
    take_profits: List[tuple] # (price, pct_to_close)
```

```

leverage: int
confidence: int
rationale: List[str]
timeframe: str
liquidation_price: float
timestamp: datetime

class SignalBot:
    """
    Production-ready Telegram bot for trading signals.
    Supports: signal alerts, portfolio tracking, risk calculator,
    admin commands, and user whitelisting.
    """

    def __init__(self, token: str, admin_ids: List[int]):
        self.token = token
        self.admin_ids = admin_ids
        self.authorized_users = set(admin_ids) # Start with admins
        self.active_signals: Dict[str, Signal] = {}
        self.user_portfolios: Dict[int, dict] = {}
        self.alert_history: List[dict] = []

        # Throttling
        self.last_alert_time: Dict[str, datetime] = {}
        self.min_alert_interval = 300 # 5 minutes between same-asset alerts

        self.app = Application.builder().token(token).build()
        self._setup_handlers()

    def _setup_handlers(self):
        """Register all command and callback handlers."""
        # Commands
        self.app.add_handler(CommandHandler("start", self.cmd_start))
        self.app.add_handler(CommandHandler("help", self.cmd_help))
        self.app.add_handler(CommandHandler("signal", self.cmd_signal))
        self.app.add_handler(CommandHandler("portfolio", self.cmd_portfolio))
        self.app.add_handler(CommandHandler("risk", self.cmd_risk_calculator))
        self.app.add_handler(CommandHandler("status", self.cmd_status))

        # Admin commands
        self.app.add_handler(CommandHandler("broadcast", self.cmd_broadcast))
        self.app.add_handler(CommandHandler("adduser", self.cmd_add_user))
        self.app.add_handler(CommandHandler("remuser", self.cmd_remove_user))
        self.app.add_handler(CommandHandler("stats", self.cmd_stats))

        # Callbacks (inline keyboard buttons)
        self.app.add_handler(CallbackQueryHandler(self.cb_signal_action))

        # Error handler
        self.app.add_error_handler(self.error_handler)

```

```

# ===== AUTH DECORATOR =====

def require_auth(self, func):
    """Decorator to check if user is authorized."""
    async def wrapper(self, update: Update, context: ContextTypes.DEFAULT_TYPE):
        user_id = update.effective_user.id
        if user_id not in self.authorized_users:
            await update.message.reply_text(
                "$$\vcenter{\hbox{\includegraphics[height=\ht\strutbox]{/usr/share/docminer/emoji/26d4.png}}}}$$ You are not authorized to use this bot.\n"
                "Contact admin for access."
            )
        return
    return await func(self, update, context)
    return wrapper

def require_admin(self, func):
    """Decorator to check if user is admin."""
    async def wrapper(self, update: Update, context: ContextTypes.DEFAULT_TYPE):
        user_id = update.effective_user.id
        if user_id not in self.admin_ids:
            await update.message.reply_text("$$\vcenter{\hbox{\includegraphics[height=\ht\strutbox]{/usr/share/docminer/emoji/26d4.png}}}}$$ Admin only command.")
        return
    return await func(self, update, context)
    return wrapper

# ===== USER COMMANDS =====

async def cmd_start(self, update: Update, context: ContextTypes.DEFAULT_TYPE):
    """Welcome message."""
    user = update.effective_user
    welcome_text = f"""
    $$\vcenter{\hbox{\includegraphics[height=\ht\strutbox]{/usr/share/docminer/emoji/1f3af.png}}}}$$
    <b>Welcome to Signal Agent Bot</b>

    Hello {user.first_name}! I provide institutional-grade trading signals for:
    • Crypto (BTC, ETH, SOL, etc.)
    • Gold (XAU/USD)

    <b>Commands:</b>
    /signal — View latest active signal
    /portfolio — Track your positions
    /risk — Risk calculator
    /status — Bot & market status
    /help — Full command list

    $$\vcenter{\hbox{\includegraphics[height=\ht\strutbox]{/usr/share/docminer/emoji/26a0-fe0f.png}}}}$$ <i>Trading involves risk. Past performance ≠ future results.</i>
    """
    await update.message.reply_text(welcome_text, parse_mode='HTML')

```

```

    async def cmd_help(self, update: Update, context: ContextTypes.DEFAULT_TYPE):
        """Help message."""
        help_text = """
$$$\\vcenter{{\\hbox{{\\includegraphics[height=\\ht\\strutbox]{{/usr/share/docminer/emoji/1f4da.png}}}}}$$$
↪ <b>Command Reference</b>

<b>Trading:</b>
/signal — Latest active signal with entry/SL/TP
/portfolio — Your open positions & P&L
/risk [balance] [entry] [sl] [leverage] — Position size calculator

<b>Market:</b>
/status — Bot uptime, active pairs, last scan

<b>Admin:</b>
/broadcast [message] — Send to all users
/adduser [user_id] — Authorize new user
/remuser [user_id] — Remove user
/stats — Signal performance stats

<i>Signals are generated by multi-factor confluence engine.</i>
"""

        await update.message.reply_text(help_text, parse_mode='HTML')

@require_auth
async def cmd_signal(self, update: Update, context: ContextTypes.DEFAULT_TYPE):
    """Show latest signal."""
    if not self.active_signals:
        await update.message.reply_text(
            """$$$\\vcenter{{\\hbox{{\\includegraphics[height=\\ht\\strutbox]{{/usr/share/docminer/emoji/
↪ 1f4ed.png}}}}}$$$ No active signals right now.\\n"
            "Markets are consolidating. I'll alert you when a setup forms."
        )
        return

    # Show most recent signal
    latest = list(self.active_signals.values())[-1]
    msg = self._format_signal_message(latest)

    keyboard = [
        [InlineKeyboardButton("$$$\\vcenter{{\\hbox{{\\includegraphics[height=\\ht\\strutbox]{{/usr/
↪ share/docminer/emoji/2705.png}}}}}$$$ I'm Taking This",
↪ callback_data=f"take_{latest.asset}"),
        InlineKeyboardButton("$$$\\vcenter{{\\hbox{{\\includegraphics[height=\\ht\\strutbox]{{/usr/share/
↪ docminer/emoji/274c.png}}}}}$$$ Skip", callback_data=f"skip_{latest.asset}")],
        [InlineKeyboardButton("$$$\\vcenter{{\\hbox{{\\includegraphics[height=\\ht\\strutbox]{{/usr/share/
↪ docminer/emoji/1f4ca.png}}}}}$$$ Details", callback_data=f"detail_{latest.asset}"),
        InlineKeyboardButton("$$$\\vcenter{{\\hbox{{\\includegraphics[height=\\ht\\strutbox]{{/usr/share/
↪ docminer/emoji/1f514.png}}}}}$$$ Alert Me at Entry",
↪ callback_data=f"alert_{latest.asset}")]]

```

```

]

await update.message.reply_text(
    msg,
    parse_mode='HTML',
    reply_markup=InlineKeyboardMarkup(keyboard)
)

@require_auth
async def cmd_portfolio(self, update: Update, context: ContextTypes.DEFAULT_TYPE):
    """Show user's portfolio."""
    user_id = update.effective_user.id
    portfolio = self.user_portfolios.get(user_id, {})

    if not portfolio.get('positions'):
        await update.message.reply_text(
            "$$\vcenter{\hbox{\includegraphics[height=\ht\strutbox]{/usr/share/docminer/emoji/1f4ed.png}}}}$$ No open positions.\nUse /signal to find setups."
        )
        return

    msg = "$$\vcenter{\hbox{\includegraphics[height=\ht\strutbox]{/usr/share/docminer/emoji/1f4ca.png}}}}$$ <b>Your Portfolio</b>\n\n"
    total_pnl = 0

    for pos in portfolio['positions']:
        pnl = pos.get('unrealized_pnl', 0)
        total_pnl += pnl
        emoji = "$$\vcenter{\hbox{\includegraphics[height=\ht\strutbox]{/usr/share/docminer/emoji/1f7e2.png}}}}$$" if pnl > 0 else
        "$$\vcenter{\hbox{\includegraphics[height=\ht\strutbox]{/usr/share/docminer/emoji/1f534.png}}}}$$" if pnl < 0 else
        "$$\vcenter{\hbox{\includegraphics[height=\ht\strutbox]{/usr/share/docminer/emoji/26aa.png}}}}$$"
        msg += f"{emoji} <b>{pos['asset']}</b> {pos['direction']}\n"
        msg += f"    Entry: ${pos['entry']:.2f} | Current: ${pos['current']:.2f}\n"
        msg += f"    P&L: {pnl:+.2f}% | Size: {pos['size']}\n\n"

    msg += f"\n<b>Total Unrealized P&L: {total_pnl:+.2f}%</b>"
    await update.message.reply_text(msg, parse_mode='HTML')

@require_auth
async def cmd_risk_calculator(self, update: Update, context: ContextTypes.DEFAULT_TYPE):
    """Interactive risk calculator."""
    args = context.args

    if len(args) < 4:
        await update.message.reply_text(
            "$$\vcenter{\hbox{\includegraphics[height=\ht\strutbox]{/usr/share/docminer/emoji/1f4b0.png}}}}$$ <b>Risk Calculator</b>\n\n"
            "Usage: /risk [balance] [entry] [stop] [leverage]\n"

```

```

        "Example: /risk 10000 67300 66800 5\n\n"
        "I'll calculate your position size, margin, and liquidation price."
    )
    return

try:
    balance = float(args[0])
    entry = float(args[1])
    stop = float(args[2])
    leverage = int(args[3])

    risk_pct = 2.0 # Default 2%
    risk_amount = balance * (risk_pct / 100)
    stop_distance = abs(entry - stop)
    position_size = risk_amount / stop_distance
    margin = (position_size * entry) / leverage

    # Liquidation (simplified, varies by exchange)
    if entry > stop: # Long
        liq = entry * (1 - (0.9 / leverage))
    else: # Short
        liq = entry * (1 + (0.9 / leverage))

    msg = f"""
    $$\vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/docminer/emoji/1f4b0.png}}}}}$$
    ↳ <b>Risk Calculation</b>

    Account Balance: ${balance:,.2f}
    Risk per Trade: {risk_pct}% (${risk_amount:,.2f})

    $$\vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/docminer/emoji/1f4ca.png}}}}}$$
    ↳ <b>Position Details:</b>
    Entry: ${entry:,.2f}
    Stop Loss: ${stop:,.2f}
    Distance: ${stop_distance:,.2f} ({stop_distance/entry*100:.2f}%)

    Position Size: {position_size:.6f} units
    Margin Required: ${margin:,.2f}
    Leverage: {leverage}x

    $$\vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/docminer/emoji/
    ↳ 26a0-fe0f.png}}}}}$$ <b>Liquidation Price: ~${liq:,.2f}</b>
    Distance to Liq: {abs(entry-liq)/entry*100:.2f}%

    <i>Margin is {margin/balance*100:.1f}% of your account.</i>
    """

    await update.message.reply_text(msg, parse_mode='HTML')

except ValueError:

```

```

        await update.message.reply_text(f"$${\vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/docminer/emoji/274c.png}}}}}}$$ Invalid numbers. Use: /risk 10000 67300 66800 5")

@require_auth
async def cmd_status(self, update: Update, context: ContextTypes.DEFAULT_TYPE):
    """Bot status."""
    uptime = datetime.now() - getattr(self, 'start_time', datetime.now())
    msg = f"""
    $$\vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/docminer/emoji/1f916.png}}}}}}$$
    ↳ <b>Bot Status</b>

    Uptime: {uptime}
    Active Signals: {len(self.active_signals)}
    Authorized Users: {len(self.authorized_users)}
    Markets Scanned: BTC, ETH, SOL, XAU/USD

    Last Scan: {datetime.now().strftime('%H:%M:%S')}
    Next Scan: Every 1H + volatility triggers
    """
    await update.message.reply_text(msg, parse_mode='HTML')

# ===== ADMIN COMMANDS =====

@require_admin
async def cmd_broadcast(self, update: Update, context: ContextTypes.DEFAULT_TYPE):
    """Broadcast message to all users."""
    if not context.args:
        await update.message.reply_text("Usage: /broadcast Your message here")
        return

    message = " ".join(context.args)
    sent = 0
    failed = 0

    for user_id in self.authorized_users:
        try:
            await context.bot.send_message(user_id,
                ↳ f"$${\vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/docminer/emoji/1f4e2.png}}}}}}$$ <b>Broadcast</b>\n\n{message}", parse_mode='HTML')
            sent += 1
        except Exception:
            failed += 1

    await update.message.reply_text(f"$${\vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/docminer/emoji/2705.png}}}}}}$$ Sent to {sent} users. Failed: {failed}")

@require_admin
async def cmd_add_user(self, update: Update, context: ContextTypes.DEFAULT_TYPE):
    """Add authorized user."""
    if not context.args:

```

```

        await update.message.reply_text("Usage: /adduser 123456789")
        return

    try:
        user_id = int(context.args[0])
        self.authorized_users.add(user_id)
        await
        ↪ update.message.reply_text(f"${vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/
        ↪ usr/share/docminer/emoji/2705.png}}}}}}${ User {user_id} authorized.")
    except ValueError:
        await update.message.reply_text("${vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/
        ↪ usr/share/docminer/emoji/274c.png}}}}}}${ Invalid user ID.")

@require_admin
async def cmd_remove_user(self, update: Update, context: ContextTypes.DEFAULT_TYPE):
    """Remove user."""
    if not context.args:
        await update.message.reply_text("Usage: /remuser 123456789")
        return

    try:
        user_id = int(context.args[0])
        self.authorized_users.discard(user_id)
        await
        ↪ update.message.reply_text(f"${vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/
        ↪ usr/share/docminer/emoji/2705.png}}}}}}${ User {user_id} removed.")
    except ValueError:
        await update.message.reply_text("${vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/
        ↪ usr/share/docminer/emoji/274c.png}}}}}}${ Invalid user ID.")

@require_admin
async def cmd_stats(self, update: Update, context: ContextTypes.DEFAULT_TYPE):
    """Signal performance stats."""
    total = len(self.alert_history)
    if total == 0:
        await update.message.reply_text("${vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/
        ↪ usr/share/docminer/emoji/1f4ca.png}}}}}}${ No signal history yet.")
        return

    wins = sum(1 for a in self.alert_history if a.get('result') == 'win')
    win_rate = wins / total * 100

    msg = f"""
    ${vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/docminer/emoji/1f4ca.png}}}}}}${
    ↪ <b>Signal Performance</b>

    Total Signals: {total}
    Wins: {wins}
    Losses: {total - wins}
    Win Rate: {win_rate:.1f}%

```

```

<i>Track results by marking signals win/loss.</i>
"""
    await update.message.reply_text(msg, parse_mode='HTML')

# ===== CALLBACK HANDLERS =====

async def cb_signal_action(self, update: Update, context: ContextTypes.DEFAULT_TYPE):
    """Handle inline button clicks."""
    query = update.callback_query
    await query.answer()

    action, asset = query.data.split("_", 1)
    user_id = update.effective_user.id

    if action == "take":
        # User is taking the trade
        await query.edit_message_text(
            query.message.text + "\n\n$$\vcenter{\hbox{\includegraphics[height=\ht\strutbox]{{/usr/share/docminer/emoji/2705.png}}}}$$ <b>You marked this as TAKEN</b>\n"
            ↪ "Set your alerts and manage risk. Good luck!",
            parse_mode='HTML'
        )
        # Log to portfolio
        if user_id not in self.user_portfolios:
            self.user_portfolios[user_id] = {'positions': []}

    elif action == "skip":
        await query.edit_message_text(
            query.message.text + "\n\n$$\vcenter{\hbox{\includegraphics[height=\ht\strutbox]{{/usr/share/docminer/emoji/274c.png}}}}$$ <b>Skipped</b>",
            ↪ parse_mode='HTML'
        )

    elif action == "detail":
        signal = self.active_signals.get(asset)
        if signal:
            detail = f"""
            $$\vcenter{\hbox{\includegraphics[height=\ht\strutbox]{{/usr/share/docminer/emoji/1f4ca.png}}}}$$
            ↪ <b>{asset}</b> Full Analysis</b>

            <b>Confluence Factors:</b>
            {chr(10).join(f"• {f}" for f in signal.rationale)}

            <b>Risk Metrics:</b>
            R:R Ratio: 1:{(signal.take_profits[0][0] - signal.entry) / (signal.entry - signal.stop_loss):.1f}
            Confidence: {signal.confidence}%
            Timeframe: {signal.timeframe}

            <b>Management:</b>
            • Move SL to breakeven after TP1 hits
            • Trail remaining 25% with ATR-based stop

```

- Close full position if structure breaks

```

"""
    await context.bot.send_message(user_id, detail, parse_mode='HTML')

elif action == "alert":
    await context.bot.send_message(
        user_id,
        f"${vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/docminer/emoji/
        ↳ 1f514.png}}}}}}$<b>Alert Set</b>\n\nI'll notify you when {asset} enters the
        ↳ zone.",
        parse_mode='HTML'
    )

# ===== SIGNAL EMISSION =====

def _format_signal_message(self, signal: Signal) -> str:
    """Format signal into beautiful Telegram message."""
    direction_emoji = "${vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/
    ↳ docminer/emoji/1f7e2.png}}}}}}$<b> if signal.direction == "LONG" else
    ↳ "${vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/docminer/emoji/
    ↳ 1f534.png}}}}}}$<b>"

    tps_text = "\n".join([
        f"${vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/docminer/emoji/
        ↳ 1f3af.png}}}}}}$ TP{i+1}: ${tp[0]:,.2f} (Close {tp[1]})%"
        for i, tp in enumerate(signal.take_profits)
    ])

    rationale_text = "\n".join([f"${vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/
    ↳ share/docminer/emoji/2713.png}}}}}}$<b> {r}" for r in signal.rationale[5]])

    msg = f"""
<b>direction_emoji</b> <b>SIGNAL: {signal.asset} {signal.direction}</b>



---



${vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/docminer/emoji/1f3af.png}}}}}}$<b>
↳ <b>Entry Zone:</b> ${signal.entry_zone[0]:,.2f} - ${signal.entry_zone[1]:,.2f}
${vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/docminer/emoji/26d4.png}}}}}}$<b>
↳ <b>Stop Loss:</b> ${signal.stop_loss:,.2f}
{tps_text}



---



${vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/docminer/emoji/
↳ 2699-fe0f.png}}}}}}$<b>Leverage:</b> {signal.leverage}x
${vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/docminer/emoji/1f4b0.png}}}}}}$<b>
↳ <b>Confidence:</b> {signal.confidence}%
${vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/docminer/emoji/
↳ 23f1-fe0f.png}}}}}}$<b>Timeframe:</b> {signal.timeframe}

```

```

    $$$\vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/docminer/emoji/1f9e0.png}}}}}}$$$
    ↪ <b>Rationale:</b>
    {rationale_text}



---



    $$$\vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/docminer/emoji/
    ↪ 26a0-fe0f.png}}}}}}$$$ <b>Liquidation:</b> ~${signal.liquidation_price:,.2f}
    <i>Distance to liquidation: {abs(signal.entry - signal.liquidation_price) / signal.entry * 100:.1f}%</i>

    $$$\vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/docminer/emoji/1f550.png}}}}}}$$$
    ↪ <i>Signal time: {signal.timestamp.strftime('%H:%M UTC')}</i>
        """
        return msg

    async def emit_signal(self, signal: Signal, context: Optional[ContextTypes.DEFAULT_TYPE] = None):
        """
        Send signal to all authorized users.
        Includes throttling to prevent spam.
        """
        # Throttle check
        now = datetime.now()
        last_time = self.last_alert_time.get(signal.asset)
        if last_time and (now - last_time).seconds < self.min_alert_interval:
            logger.info(f"Throttled signal for {signal.asset}")
            return

        self.last_alert_time[signal.asset] = now
        self.active_signals[signal.asset] = signal
        self.alert_history.append({
            'asset': signal.asset,
            'direction': signal.direction,
            'timestamp': now,
            'result': None
        })

        msg = self._format_signal_message(signal)

        keyboard = [
            [InlineKeyboardButton("$$$$\vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/
            ↪ docminer/emoji/2705.png}}}}}}$$$ Take Trade", callback_data=f"take_{signal.asset}"),
            InlineKeyboardButton("$$$$\vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/
            ↪ docminer/emoji/274c.png}}}}}}$$$ Skip", callback_data=f"skip_{signal.asset}")),
            [InlineKeyboardButton("$$$$\vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/
            ↪ share/docminer/emoji/1f4ca.png}}}}}}$$$ Full Details",
            ↪ callback_data=f"detail_{signal.asset}")]]
        ]

        sent_count = 0
        for user_id in self.authorized_users:
            try:

```

```

        if context:
            await context.bot.send_message(
                user_id, msg, parse_mode='HTML',
                reply_markup=InlineKeyboardMarkup(keyboard)
            )
        else:
            # Use application bot instance
            await self.app.bot.send_message(
                user_id, msg, parse_mode='HTML',
                reply_markup=InlineKeyboardMarkup(keyboard)
            )
        sent_count += 1
    except Exception as e:
        logger.error(f"Failed to send to {user_id}: {e}")

    logger.info(f"Signal sent to {sent_count} users for {signal.asset}")

async def error_handler(self, update: object, context: ContextTypes.DEFAULT_TYPE):
    """Log errors."""
    logger.error(f"Update {update} caused error: {context.error}")

# ===== LIFECYCLE =====

async def start(self):
    """Start the bot."""
    self.start_time = datetime.now()
    logger.info("Starting Signal Bot...")
    await self.app.initialize()
    await self.app.start()

    # Use polling (for development) or webhook (for production)
    await self.app.updater.start_polling(drop_pending_updates=True)
    logger.info("Bot is running with polling.")

async def stop(self):
    """Stop the bot gracefully."""
    await self.app.updater.stop()
    await self.app.stop()
    await self.app.shutdown()
    logger.info("Bot stopped.")

# ===== SETUP & RUN =====

"""
SETUP INSTRUCTIONS:

1. Create bot with @BotFather on Telegram
   - Send /newbot
   - Name your bot
   - Save the token

```

2. Set environment variables:

```
export TELEGRAM_BOT_TOKEN="your_token_here"
export ADMIN_IDS="123456789,987654321"
```

3. Run:

```
python bot.py
```

4. For production webhook (recommended):

- Deploy to VPS/cloud
- Set webhook URL
- Use nginx reverse proxy with SSL

```
"""
```

```
async def main():
    import os
    TOKEN = os.getenv("TELEGRAM_BOT_TOKEN", "YOUR_BOT_TOKEN")
    ADMIN_IDS = [int(x) for x in os.getenv("ADMIN_IDS", "123456789").split(",")]

    bot = SignalBot(TOKEN, ADMIN_IDS)
    await bot.start()

    # Keep running
    try:
        while True:
            await asyncio.sleep(1)
    except KeyboardInterrupt:
        await bot.stop()

# asyncio.run(main())
```

3.3 Webhook Setup (Production)

```
# webhook_server.py — For production deployment
from telegram.ext import Application
import asyncio
from aiohttp import web

async def webhook_handler(request):
    """Handle incoming webhook updates."""
    application = request.app['telegram_app']
    data = await request.json()
    update = Update.de_json(data, application.bot)
    await application.process_update(update)
    return web.Response()

async def setup_webhook():
    TOKEN = os.getenv("TELEGRAM_BOT_TOKEN")
    WEBHOOK_URL = os.getenv("WEBHOOK_URL") # https://yourdomain.com/webhook
```

```

PORT = int(os.getenv("PORT", 8443))

application = Application.builder().token(TOKEN).build()

# Set webhook
await application.bot.set_webhook(WEBHOOK_URL)

# Setup aiohttp server
app = web.Application()
app['telegram_app'] = application
app.router.add_post('/webhook', webhook_handler)

runner = web.AppRunner(app)
await runner.setup()
site = web.TCPSite(runner, '0.0.0.0', PORT)
await site.start()

print(f"Webhook server running on port {PORT}")

while True:
    await asyncio.sleep(3600)

# asyncio.run(setup_webhook())

```

PART 4: COMPLETE SYSTEM INTEGRATION

4.1 Main Orchestrator

```

class SignalAgentOrchestrator:
    """
    Ties everything together:
    - Fetches data from exchanges
    - Runs confluence analysis
    - Emits signals via Telegram
    - Manages risk checks
    - Tracks performance
    """

    def __init__(self):
        self.exchange = ExchangeManager(
            exchange_id='binance',
            api_key=os.getenv('BINANCE_API_KEY'),
            api_secret=os.getenv('BINANCE_SECRET'),
            sandbox=True
        )
        self.confluence = ConfluenceEngine()
        self.bot = SignalBot(
            token=os.getenv('TELEGRAM_BOT_TOKEN'),
            admin_ids=[int(x) for x in os.getenv('ADMIN_IDS', '').split(',') if x]
        )

```

```

self.risk_engine = RiskEngine(account_balance=10000)

self.assets = ['BTC/USDT:USDT', 'ETH/USDT:USDT', 'SOL/USDT:USDT']
self.gold_provider = GoldDataProvider()

self.scan_interval = 3600 # 1 hour
self.last_scan = 0

async def run(self):
    """Main loop."""
    await self.bot.start()
    await self.exchange.start_websocket(self.assets)

    while True:
        try:
            await self.scan_all_markets()
            await asyncio.sleep(self.scan_interval)
        except Exception as e:
            logger.error(f"Main loop error: {e}")
            await asyncio.sleep(60)

async def scan_all_markets(self):
    """Scan all configured markets for signals."""
    logger.info("Starting market scan...")

    for asset in self.assets:
        try:
            # Fetch multi-timeframe data
            ohlcv_4h = await self.exchange.fetch_ohlcv(asset, '4h', 200)
            ohlcv_1h = await self.exchange.fetch_ohlcv(asset, '1h', 200)

            df_4h = pd.DataFrame(ohlcv_4h, columns=['timestamp', 'open', 'high', 'low', 'close',
            ↪ 'volume'])
            df_1h = pd.DataFrame(ohlcv_1h, columns=['timestamp', 'open', 'high', 'low', 'close',
            ↪ 'volume'])

            # Fetch market data
            funding = await self.exchange.fetch_funding_rate(asset)
            oi = await self.exchange.fetch_open_interest(asset)

            current_price = self.exchange.price_cache.get(asset.replace('/', '').replace(':', ''))
            ↪ {}).get('price')
            if not current_price:
                ticker = await self.exchange.fetch_ticker(asset)
                current_price = ticker['last']

            # Calculate OI change
            # (Would need historical OI for proper calculation)
            oi_change = 0

            # Run confluence analysis

```

```

        result = self.confluence.analyze(
            df_4h, df_1h, current_price,
            funding.get('fundingRate', 0),
            oi_change
        )

    if result['valid']:
        # Risk check
        risk_result = self.risk_engine.calculate_position(
            result['entry'], result['stop_loss'], result.get('leverage', 5)
        )

        if risk_result['valid']:
            signal = Signal(
                asset=asset,
                direction=result['direction'],
                entry=result['entry'],
                entry_zone=(result['entry'] * 0.998, result['entry'] * 1.002),
                stop_loss=result['stop_loss'],
                take_profits=result['take_profits'],
                leverage=result.get('leverage', 5),
                confidence=result['confidence'],
                rationale=result['factors'],
                timeframe=result['timeframe'],
                liquidation_price=risk_result['liquidation'],
                timestamp=datetime.now()
            )

            await self.bot.emit_signal(signal)
            logger.info(f"Signal emitted: {asset} {result['direction']}")

    except Exception as e:
        logger.error(f"Error scanning {asset}: {e}")

# Scan Gold
await self.scan_gold()

self.last_scan = time.time()
logger.info("Market scan complete.")

async def scan_gold(self):
    """Scan XAU/USD separately."""
    try:
        df = await self.gold_provider.fetch_xau_usd('H1', 500)
        # Similar confluence analysis adapted for gold
        # (Simplified - would need same multi-TF approach)
        logger.info("Gold scan complete.")
    except Exception as e:
        logger.error(f"Gold scan error: {e}")

```

```
# Run everything
async def main():
    orchestrator = SignalAgentOrchestrator()
    await orchestrator.run()

# asyncio.run(main())
```

APPENDIX: Requirements & Environment

```
# requirements.txt
ccxt>=4.2.0
pandas>=2.0.0
numpy>=1.24.0
python-telegram-bot>=20.0
websockets>=12.0
aiohttp>=3.9.0
scipy>=1.11.0
yfinance>=0.2.0
python-dotenv>=1.0.0
```

```
# .env file
BINANCE_API_KEY=your_testnet_key
BINANCE_SECRET=your_testnet_secret
TELEGRAM_BOT_TOKEN=your_bot_token
ADMIN_IDS=123456789,987654321
OANDA_TOKEN=your_oanda_token
```

FINAL WARNINGS

1. **NEVER** run with real money until backtested for 6+ months.
2. Start with 1% risk per trade, not 2%.
3. Use testnet/sandbox for 3 months minimum.
4. Keep a trading journal. The bot is only as good as your discipline.
5. Markets change. What works in bull markets fails in bear markets. Adapt or die.